

Part A – Theory Questions

1. Define Unsupervised Learning. Explain how it differs from supervised learning.

Unsupervised Learning is a machine learning technique that analyzes unlabeled data to discover hidden patterns, structures, or relationships without predefined output labels. Its main objective is knowledge discovery rather than prediction. Major techniques include clustering and association rule learning.

Difference between Supervised and Unsupervised Learning

Aspect	Supervised Learning	Unsupervised Learning
Data Type	Labeled data	Unlabeled data
Objective	Predict output	Discover hidden structure
Output	Class or value	Groups or association rules
Examples	Classification, Regression	Clustering, Association

In supervised learning, the model learns from known outputs, whereas in unsupervised learning, the model identifies patterns automatically from the data.

2. Explain the main objectives of unsupervised learning.

The major objectives of unsupervised learning are:

1. **Discover hidden patterns** in unlabeled data.
2. **Group similar data points** using clustering techniques.
3. **Identify relationships** among variables using association rules.
4. **Reduce data complexity** through dimensionality reduction.
5. **Detect anomalies or outliers** in datasets.
6. **Support decision making** by extracting meaningful information from raw data.

Applications include customer segmentation, fraud detection, image processing, document clustering, and bioinformatics.

3. Define clustering and explain its importance in machine learning.

Clustering is the process of grouping objects such that objects within the same cluster are highly similar, while objects belonging to different clusters are highly dissimilar. It is an unsupervised learning technique.

Importance of Clustering

1. Helps discover hidden patterns in data.
2. Reduces complexity of large datasets.
3. Used in customer segmentation and recommendation systems.
4. Supports anomaly detection.
5. Useful in text mining, image processing, and market analysis.

Clustering improves understanding of unlabeled data and serves as a foundation for other machine learning techniques.

4. Explain the K-Means clustering algorithm with step-by-step procedure.

K-Means is a centroid-based partitioning clustering algorithm in which each cluster is represented by its centroid (mean point). The number of clusters K must be predefined.

Steps of K-Means Algorithm

1. Select K initial centroids randomly.
2. Assign each data point to the nearest centroid.
3. Recompute centroids by calculating the mean of points in each cluster.
4. Reassign data points to the updated centroids.
5. Repeat steps 3 and 4 until centroids stop changing significantly.
6. Output final clusters and centroids.

Advantages

- Simple and fast.
- Efficient for large datasets.

Limitations

- Sensitive to outliers.
- Requires predefined K.
- Cannot handle non-spherical clusters effectively.

5. Discuss advantages and limitations of clustering techniques.

Advantages

1. Discovers hidden patterns in unlabeled data.
2. Reduces data complexity.
3. Useful in customer segmentation and anomaly detection.
4. Helps organize large datasets efficiently.
5. Supports recommendation systems and market analysis.

Limitations

1. Difficult to determine the optimal number of clusters.
2. Sensitive to noise and outliers.
3. Different algorithms may produce different clusters.
4. Computational cost may increase for large datasets.
5. Some methods fail for non-spherical clusters.

Thus, clustering is useful for pattern discovery but requires careful algorithm selection.

6. Explain the difference between hierarchical clustering and partition clustering.

Hierarchical Clustering	Partition Clustering
Builds hierarchy of clusters	Divides data into non-overlapping clusters
Represented using dendrogram	Represented using centroids/medoids
Does not require predefined K initially	Requires predefined K
Uses agglomerative/divisive approach	Uses iterative partitioning
More computationally expensive	Faster and scalable

Examples

- Hierarchical: Agglomerative Clustering
- Partitioning: K-Means, K-Medoids

Hierarchical clustering is suitable for nested structures, whereas partition clustering is efficient for large datasets.

7. What is Association Rule Mining?

Association Rule Mining is a technique used to discover interesting relationships and patterns among variables in large datasets. It is commonly used in market basket analysis to identify items frequently purchased together.

Example:

$\{\text{Bread, Milk}\} \rightarrow \{\text{Egg}\}$

This rule indicates that customers who buy Bread and Milk are likely to buy Egg. Association rule mining is widely used in retail, medical diagnosis, and web mining.

8. Explain the concepts of support and confidence in association rules.

Support

Support measures how frequently an itemset appears in the transaction dataset.

$$\text{Support}(X \rightarrow Y) = \frac{\text{SupportCount}(X \cup Y)}{N}$$

It indicates the proportion of transactions containing both X and Y.

Confidence

Confidence measures the reliability of an association rule.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{SupportCount}(X \cup Y)}{\text{SupportCount}(X)}$$

It represents the probability that Y occurs when X occurs.

Example:

For the rule Bread $\rightarrow$ Milk,

Support = 0.667

Confidence = 0.80

9. Describe the Apriori algorithm used for association rule mining.

Apriori is a popular algorithm used to generate frequent itemsets and association rules from transaction data. It follows a bottom-up approach.

Apriori Principle

If an itemset is frequent, all its subsets must also be frequent. If an itemset is infrequent, all its supersets are also infrequent.

Steps of Apriori Algorithm

1. Generate frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent 1-itemsets.
3. Prune infrequent itemsets using minimum support.
4. Repeat iteratively for k-itemsets.
5. Generate strong association rules using minimum confidence.

Advantages

- Simple and effective.
- Reduces search space using pruning.

Limitations

- Requires multiple database scans.
- Computationally expensive for large datasets.

10. Give a real-world example where association rules are used.

Association rules are widely used in Market Basket Analysis in supermarkets and e-commerce websites.

Example:

If customers frequently buy Bread and Milk together, the retailer may place these products nearby or recommend one product when the other is purchased.

Applications include:

- Product recommendation systems
- Cross-selling
- Medical diagnosis
- Web usage mining

This helps businesses improve sales and customer experience.

11. Compare biological neuron and artificial neuron.

Biological Neuron	Artificial Neuron
Natural nerve cell	Mathematical model
Dendrites receive signals	Inputs receive signals
Synapses determine strength	Weights determine importance
Soma processes signals	Summation function processes inputs
Axon transmits output	Output node transmits result
Learns biologically	Learns computationally

Artificial neurons are inspired by biological neurons and form the basis of Artificial Neural Networks (ANN).

12. Explain the mathematical model of an artificial neuron.

An artificial neuron receives multiple inputs and multiplies them with corresponding weights. The weighted inputs are summed along with bias and passed through an activation function.

Mathematical model:

$$y = f(\sum_{i=1}^n w_i x_i + b)$$

Where:

- x_i = inputs
- w_i = weights
- b = bias
- f = activation function
- y = output

Weights represent importance of inputs, while bias shifts the activation threshold.

13. Define activation function and explain different activation functions.

An activation function introduces non-linearity into neural networks and determines whether a neuron should activate.

(a) Binary Step Function

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Produces binary output.

(b) Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-kx}}$$

Output range is (0,1). Used in binary classification.

(c) Tanh Function

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Output range is (-1,1).

(d) ReLU Function

$$f(x) = \max(0, x)$$

Most widely used activation function in deep learning.

14. Explain the McCulloch–Pitts neuron model. Discuss assumptions and limitations.

The McCulloch–Pitts neuron model was the first artificial neuron model proposed in 1943. It uses binary inputs and outputs with threshold logic.

Working

- Inputs are multiplied by weights.
- Weighted sum is calculated.
- If the sum exceeds threshold, output becomes 1; otherwise 0.

Assumptions

1. Inputs are binary.
2. Outputs are binary.
3. Fixed threshold value.
4. No learning capability.

Limitations

1. Cannot learn from data.
2. Works only for simple logical operations.
3. Cannot solve complex non-linear problems.

15. Explain the Perceptron model. Discuss perceptron learning rule.

Perceptron is the first learning neural model introduced by Rosenblatt in 1958. It performs binary classification for linearly separable data.

Learning Rule

$$w_{new} = w_{old} + \eta(t - y)x$$

Where:

- η = learning rate
- t = target output
- y = predicted output
- x = input

Features

- Adjusts weights using error correction.
- Learns iteratively from mistakes.
- Cannot solve XOR problem.

16. Explain ADALINE. Difference between Perceptron and ADALINE.

ADALINE (Adaptive Linear Neuron) was developed by Bernard Widrow. It uses the Least Mean Square (LMS) learning rule and produces continuous outputs.

Formula

$$y = \sum w_i x_i + b$$

Difference between Perceptron and ADALINE

Perceptron	ADALINE
Uses threshold output	Uses continuous output

Perceptron

Learning based on classification error

Binary activation

Less accurate

ADALINE

Learning based on LMS error

Linear activation

More stable learning

17. Describe neural network architectures.

(a) Single Layer Neural Network

- Contains input and output layer only.
- No hidden layer.
- Suitable for simple problems.

(b) Multi-Layer Neural Network

- Contains one or more hidden layers.
- Solves complex non-linear problems.

(c) Feed-Forward Neural Network

- Signals move only forward.
- No feedback loops.

(d) Recurrent Neural Network

- Contains feedback connections.
- Suitable for sequence and time-series data.

These architectures differ based on signal flow and hidden layer arrangement.

18. Explain the learning process in Artificial Neural Networks.

Learning in ANN means adjusting weights to minimize error and improve prediction accuracy.

Steps in Learning Process

1. Initialize weights randomly.
2. Feed input data into the network.
3. Compute output using activation functions.
4. Compare predicted output with target output.
5. Calculate error.
6. Adjust weights iteratively.
7. Repeat until stopping criterion is reached.

Stopping criteria:

- Error becomes sufficiently small.
 - Maximum iterations reached.
-

19. Explain the role of weights, bias and learning rate.

Weights

- Represent importance of inputs.
- Positive weights increase influence.
- Negative weights decrease influence.

Bias

- Shifts activation threshold.
- Similar to intercept in regression.

Learning Rate

- Controls size of weight updates during learning.
- Small learning rate gives slow learning.
- Large learning rate may overshoot optimal solution.

These parameters control the learning behavior of neural networks.

20. Explain the Backpropagation algorithm used for training neural networks.

Backpropagation is a supervised learning algorithm used for training multi-layer feed-forward neural networks. It adjusts weights to minimize output error.

Phases of Backpropagation

1. Forward Pass

- Inputs move from input layer to output layer.
- Activation functions generate outputs.

2. Backward Pass

- Output error is calculated.
- Errors are propagated backward.
- Weights are updated using gradient descent.

Steps

1. Initialize weights.
2. Perform forward propagation.
3. Compute output error.
4. Propagate error backward.
5. Update weights.
6. Repeat for multiple epochs.

Advantages

- Learns complex non-linear relationships.
- High accuracy.

Limitation

- Computationally expensive for deep networks.

1. Generate synthetic datasets for clustering.

```
# Generate Synthetic Dataset using make_blobs

import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate dataset with 3 clusters
X, y = make_blobs(n_samples=300,
                  centers=3,
                  cluster_std=1.0,
                  random_state=42)

# Scatter plot
plt.scatter(X[:,0], X[:,1], c=y)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Synthetic Dataset using make_blobs")
plt.grid(True)
plt.show()
```

Changing the value of centers changes the number of clusters formed.

2. Determine optimal number of clusters using Elbow Method.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300,
                  centers=4,
                  random_state=42)

wcss = []

for k in range(1,10):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    wcss.append(km.inertia_)

plt.plot(range(1,10), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid(True)
plt.show()
```

The elbow point indicates the optimal value of K.

3. Perform clustering using K-Means.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```

X, y = make_blobs(n_samples=300,
                  centers=3,
                  random_state=42)

kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels)

plt.scatter(
    kmeans.cluster_centers_[0],
    kmeans.cluster_centers_[1],
    marker='x'
)

plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()

```

Changing K from 2 to 5 changes cluster formation and centroids.

4. Hierarchical Clustering.

```

import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from scipy.cluster.hierarchy import dendrogram, linkage

X, y = make_blobs(n_samples=100,
                  centers=3,
                  random_state=42)

Z = linkage(X, method='ward')

dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

```

Hierarchical clustering forms tree-like structures unlike K-Means.

5. Find patterns using association rules.

```

transactions = [
    {"Bread", "Milk", "Egg"},
    {"Bread", "Milk"},
    {"Milk", "Egg"},
    {"Bread", "Egg"}
]

def support(itemset, transactions):
    return sum(1 for t in transactions if itemset.issubset(t)) /

```

```

len(transactions)

def confidence(X, Y, transactions):
    return support(X.union(Y), transactions) / support(X, transactions)

print("Support(Bread):",
      support({"Bread"}, transactions))

print("Confidence(Bread -> Milk):",
      confidence({"Bread"}, {"Milk"}, transactions))

```

Association rules discover relationships between frequently occurring items.

6. Implement Artificial Neuron Calculation.

```

import math
import numpy as np

x = np.array([1, 0, 1])
w = np.array([0.5, -0.6, 0.2])

y_sum = np.dot(x, w) + 0.1

# Step Function
step = 1 if y_sum >= 0 else 0

# Sigmoid Function
sigmoid = 1 / (1 + math.exp(-y_sum))

# Tanh Function
tanh = math.tanh(y_sum)

# ReLU Function
relu = max(0, y_sum)

print("Weighted Sum:", y_sum)
print("Step:", step)
print("Sigmoid:", sigmoid)
print("Tanh:", tanh)
print("ReLU:", relu)

```

7. Train a Perceptron classifier.

```

from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=200,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1
)

X_train, X_test, y_train, y_test = train_test_split(

```

```

    X, y, test_size=0.3
)

model = Perceptron(max_iter=1000, eta0=0.1)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

8. Neural Network classifier using MLPClassifier.

```

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=300,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    random_state=1
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3
)

model = MLPClassifier(
    hidden_layer_sizes=(10,5),
    activation='relu',
    learning_rate_init=0.01,
    max_iter=500
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

9. Neural Network with one hidden layer using backpropagation.

```

import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

np.random.seed(1)

weights = np.random.rand(2,1)

learning_rate = 0.1

def sigmoid(x):

```

```

        return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

for epoch in range(5000):

    output = sigmoid(np.dot(X, weights))

    error = y - output

    adjustments = learning_rate * np.dot(
        X.T,
        error * sigmoid_derivative(output)
    )

    weights += adjustments

print("Final Output:")
print(output)

```

10. Complete ML Workflow.

```

from sklearn.datasets import make_classification
from sklearn.cluster import KMeans
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset
X, y = make_classification(
    n_samples=300,
    n_features=4,
    random_state=42
)

# Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3
)

# Perceptron
p = Perceptron()
p.fit(X_train, y_train)

# Neural Network
mlp = MLPClassifier(hidden_layer_sizes=(10,5),
                    max_iter=500)

mlp.fit(X_train, y_train)

# Accuracy
print("Perceptron Accuracy:",

```

```
    accuracy_score(y_test, p.predict(X_test)))  
  
print("MLP Accuracy:",  
      accuracy_score(y_test, mlp.predict(X_test)))
```

This workflow demonstrates dataset generation, clustering, classification, and neural network training.